

Como controlar tus entornos multiagente a golpe de patrón



Roberto Navarro
Azure Evangelist



Diego Zapico
Arquitecto.

/in/dizapico
@dzapico



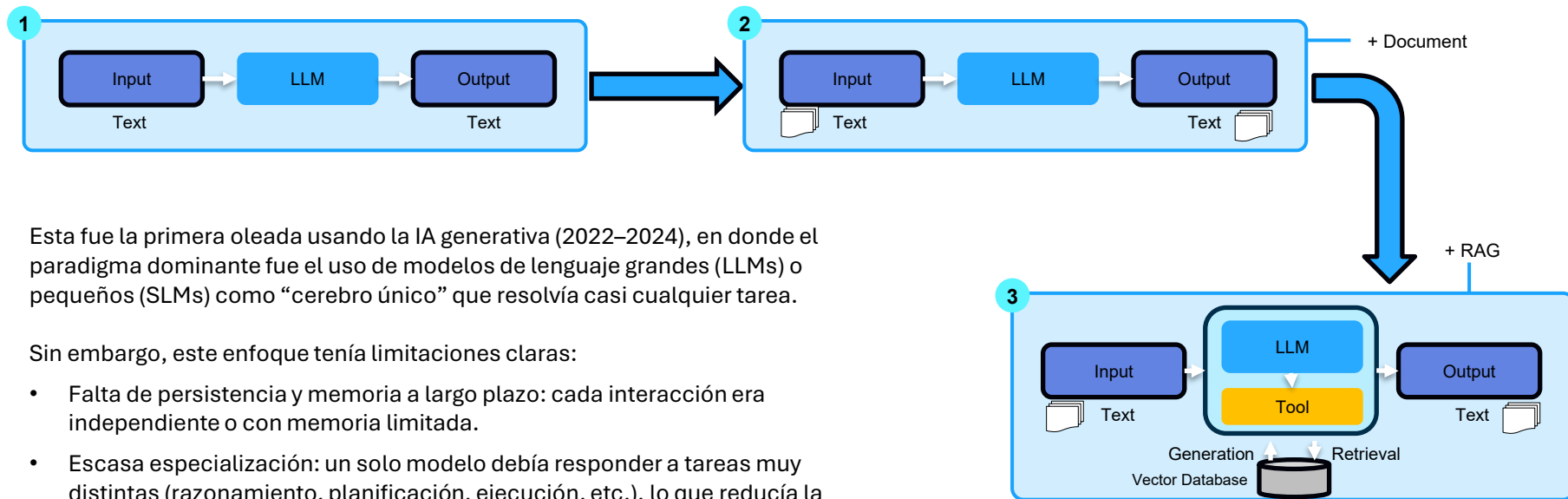
commit

Agenda

1. Introducción
2. Agentic Patterns
3. Demo

Introducción

Evolución de las arquitecturas de IA generativas



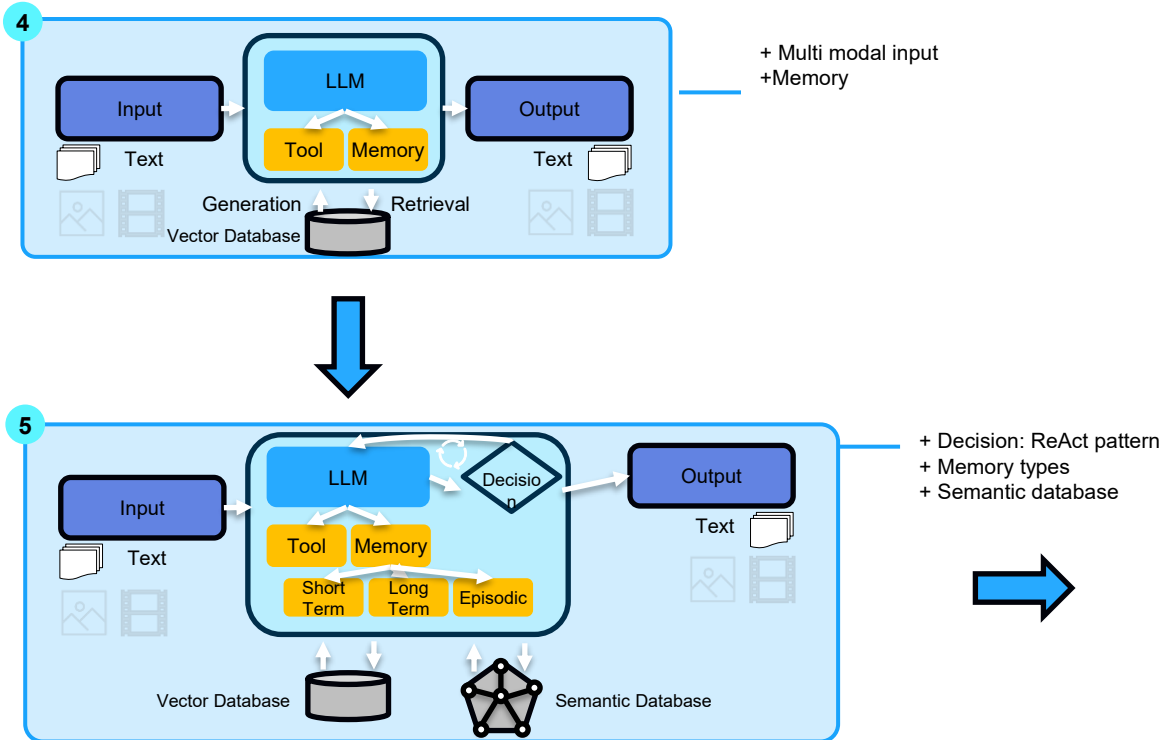
Esta fue la primera oleada usando la IA generativa (2022–2024), en donde el paradigma dominante fue el uso de modelos de lenguaje grandes (LLMs) o pequeños (SLMs) como “cerebro único” que resolvía casi cualquier tarea.

Sin embargo, este enfoque tenía limitaciones claras:

- Falta de persistencia y memoria a largo plazo: cada interacción era independiente o con memoria limitada.
- Escasa especialización: un solo modelo debía responder a tareas muy distintas (razonamiento, planificación, ejecución, etc.), lo que reducía la precisión.
- Baja capacidad de acción: el modelo solo “hablaba”, pero no actuaba ni ejecutaba tareas reales (navegar, programar, consultar APIs, etc.).
- Costes crecientes: un único modelo enorme resulta caro y poco eficiente si se usa para tareas simples.

Introducción

Evolución de las arquitecturas de IA generativas

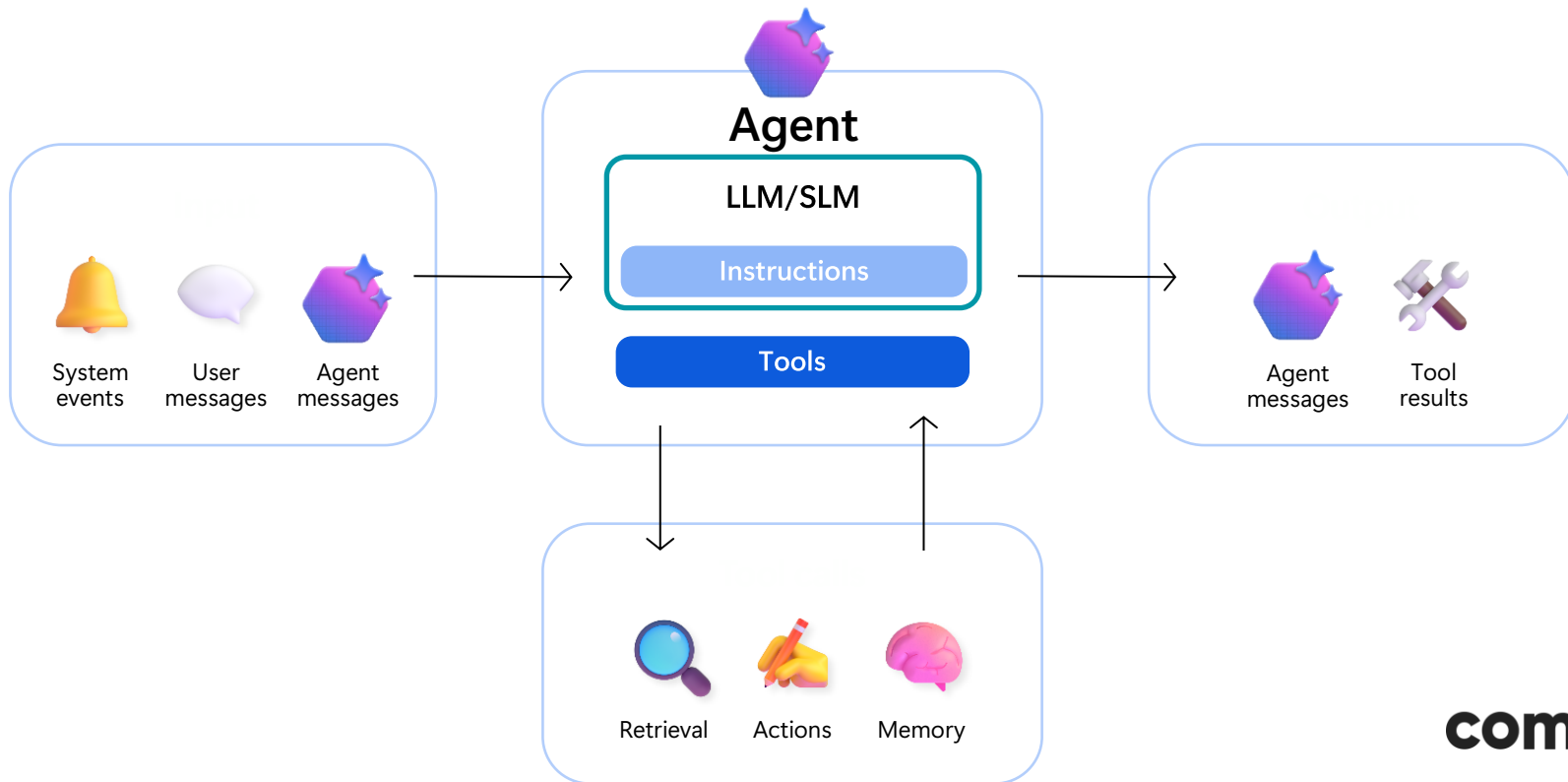


Cuando añadimos herramientas, memoria y decisión... deja de ser solo un modelo. Empieza a comportarse como un agente



Introducción

Esquema de un agente



Introducción

Características claves de un Agente Autónomo



Percepción “Entiende el mundo”

-  Sensores multimodales procesan texto, imágenes, audio y datos estructurados.
-  Filtrado contextual identifica información relevante del entorno.
-  Representación interna convierte datos en embeddings procesables.
-  Actualización continua del estado del entorno en tiempo real.



Razonamiento “Decide qué hacer”

-  Planificación estratégica descompone objetivos complejos en subtarefas.
-  Memoria persistente mantiene contexto entre interacciones.
-  Inferencia lógica aplica conocimiento previo a nuevas situaciones.
-  Toma de decisiones evalúa opciones y selecciona acciones óptimas.



Acción “Hace que algo ocurra”

-  Ejecución de herramientas interactúa con APIs, bases de datos y sistemas externos.
-  Comunicación natural genera respuestas coherentes y contextualmente apropiadas.
-  Manipulación de contenido crea, modifica y gestiona información.
-  Validación de resultados verifica la calidad y precisión de las acciones ejecutadas.

Un LLM responde.

Un agente percibe, decide y actúa.



Argentic Patterns

Agentic Patterns

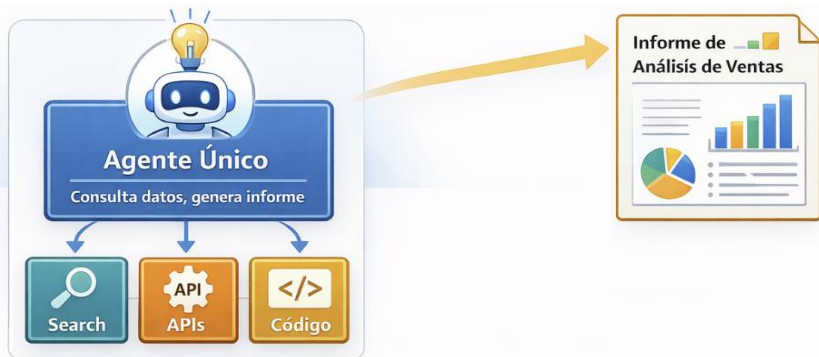
Agentes Únicos

Agente único con Herramientas

Es útil cuando tenemos tareas claras y acotadas

Se caracterizan por:

- Estar formados por un solo agente
- Usa herramientas como Bing Search, APIs, búsquedas en BB.DD.
- Hace todas las acciones: Razona, Planifica, Ejecuta y Valida.



Agentic Patterns

Agentes Complejos

Complicando nuestros agentes

Pero que ocurre cuando nuestros agentes crecen, cuando cada vez deben tener en cuenta mas cosas:

- Tareas Complejas
- Múltiples pasos
- Decisiones o dependencias
- Escalabilidad y control

Arquitecturas de Software

ANTES

Aplicación Monolítica

UI + Lógica + BD

Todo en uno

DESPUÉS



Arquitecturas Agénticas

ANTES

Agente Único

Razona
Planifica
Ejecuta

Todo en uno

DESPUÉS



Agentic Patterns

Enfoques Multiagentes

Sequential Pattern

Aquí lo que tenemos es que el ser humano ha definido explícitamente cual es el orden de ejecución de una tarea, de forma que tenemos un sistema de orquestación que va realizando las invocaciones a cada uno de los agentes:

- **Planificador/orquestados:** Se encarga de invocar a cada uno de los agentes en una pauta prefijada
- **Agentes:** realizan una tarea específica, y la secuencia de los mismos completa o resuelve la tarea global,



Agentic Patterns

Enfoques Multiagentes

Concurrent Pattern

Es útil cuando tenemos tareas independientes y acotadas

Se caracterizan por:

- Estar formadas por varias tareas sin interdependencia
- La ejecución puede venir iniciada por un proceso o por un agente.
- En el cierre puede que sea necesario unificar información o procesarla por separado.



Agentic Patterns

Enfoques Multiagentes

ReAct Pattern

El agente alterna razonamiento y las acciones de forma que se puede ir modificando el plan según sea necesario en función de los resultados:

- **Reason:** Razona cual debería ser el paso a ejecutar
- **Act:** actúa con las tolos que tiene disponible
- **Observe:** Analiza los resultados y si es lo que esperaba para continuar o replanificar.
- Repetir



Agentic Patterns

Enfoques Multiagentes

Planner & Executor

Lo que se pretende es que antes de hacer nada se planifique, y se verifique que todo tiene sentido

- **Planner:** Se encarga de generar la planificación mas adecuada
- **Executor:** Es el encargado de ejecutar las tareas planificadas

Beneficios

- Mas control
- Mejor razonamiento
- Fácil Depuración



Agentic Patterns

Enfoques Multiagentes

Reflexion & Memory Based

Se realiza un plan desde el agente planificador, y otro agente se encarga de validar esta planificación y basándose en planificaciones pasadas y sus resultados:

- **Planner:** Se encarga de generar la planificación mas adecuada
- **Evaluator:** Es el encargado de validar que la planificación es correcta:
 - Necesidades del usuario.
 - Reglas de planificación (M Procedural).
 - Planificaciones anteriores calificadas por el feedback del usuario (M Episodica).



Agentic Patterns

Enfoques Multiagentes

Magentic Pattern

En este caso tenemos un agente que se encarga de organizar y coordinar al resto de los agentes para que ejecuten sus tareas, de forma que los resultados sean los mas apropiados

- Coordina a otros agentes
- Decide a quien llamar y cuando
- Evalúa los resultados y es el responsable de dar la respuesta final al usuario



Agentic Patterns

Enfoques Multiagentes

Handoff Pattern

El **Handoff Pattern** describe un escenario en el que **un agente cede explícitamente el control de la tarea a otro agente**, pasando contexto, estado y responsabilidad.

A diferencia del patrón secuencial, aquí el traspaso **no es solo un paso del flujo**, sino un **cambio de ownership** de la tarea.

No es “ejecuta el siguiente paso”, es **“a partir de aquí, tú te haces cargo”**.



Agentic Patterns

Enfoques Multiagentes

Patrón Multi-agente colaborativa

El sistema implica múltiples agentes independientes, cada uno impulsado por modelos de lenguaje, que trabajan juntos de manera coordinada. Cada agente tiene una personalidad o rol distinto, con el contexto moldeado por indicaciones de un modelo de lenguaje específico. Estos agentes utilizan diversas herramientas para llevar a cabo sus tareas asignadas

- Separación de tareas
- Modularidad
- Diversidad
- Reusabilidad



Agentic Patterns

Enfoques Multiagentes

Arquitectura agentic combinada

1 Entrada (Usuario / Tarea)

Objetivo y contexto → estado de tarea compartido

2 Planner Agent

Genera plan, tareas y herramientas

3 Evaluación del plan + Memoria

Valida con reglas, histórico y feedback
→ aceptar o replanificar

4 Executor / Orquestador

Coordina la ejecución y decide qué agentes activar

5 Ejecución multiagente (Chat Room)

Analista · Desarrollador · Revisor
Colaboración al mismo nivel

6 ReAct en la ejecución

Reason → Act → Observe con herramientas

7 Evaluación final + Memoria

Aprende del resultado y mejora futuros planes



Agentic Patterns

Enfoques Multiagentes

Los agentes nos permiten pasar de simples respuestas a **sistemas capaces de razonar, decidir y actuar**, pero cuando la complejidad crece, un solo agente deja de ser suficiente.

Los **patrones agénticos** nos dan la estructura necesaria para escalar: desde agentes simples con herramientas, pasando por planificación, ejecución, reflexión y colaboración entre agentes especializados, hasta llegar a sistemas orquestados preparados para producción.

La clave **no está solo en el modelo**, sino en **cómo organizamos, coordinamos y hacemos colaborar a los agentes** para obtener resultados más robustos, fiables y adaptativos.





Demo

Microsoft Agent Framework

Microsoft Agent Framework

De donde venimos

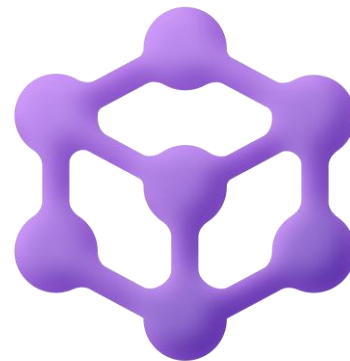


AutoGen
Research



Semantic Kernel
Production Ready

Unified Agent
Development

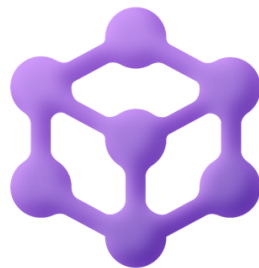
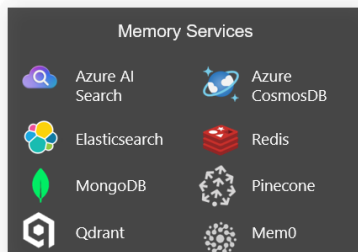
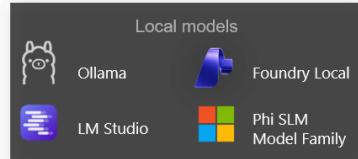
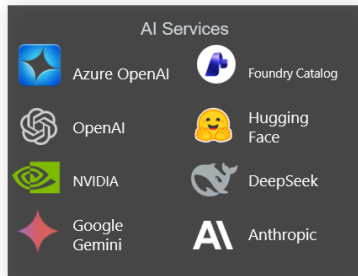


Microsoft Agent
Framework

commit

Microsoft Agent Framework

Donde estamos



Microsoft Agent Framework

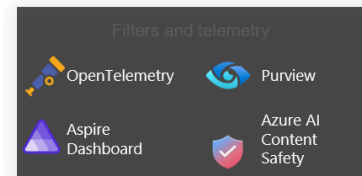
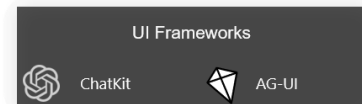
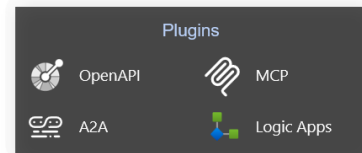
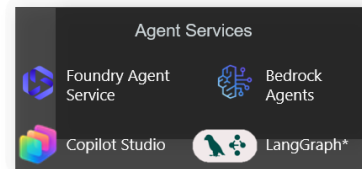
AI and Agent Orchestration



.NET



Python

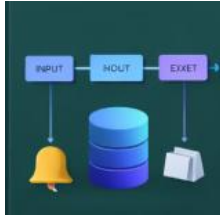


commit

Microsoft Agent Framework

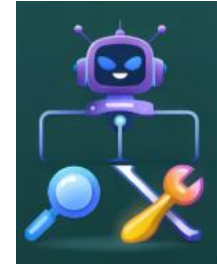
Gestion de agentes

MAF estructura, gobierna y estandariza el comportamiento del agente a lo largo de todo su ciclo de vida.



- Gestiona el ciclo de vida del agente

- Orquesta tools



- Gestiona memorias y contextos

- Permite observabilidad y control



Microsoft Agent Framework

Workflows

Un Workflow define cómo se coordinan y ejecutan en flujos controlados los agentes.

Workflows Predefinidos

Patrones listos para usar
que aceleran escenarios comunes.

- Secuenciales
- Concurrente
- Handoff
- Chat grupales
- Magentic

Workflows personalizables

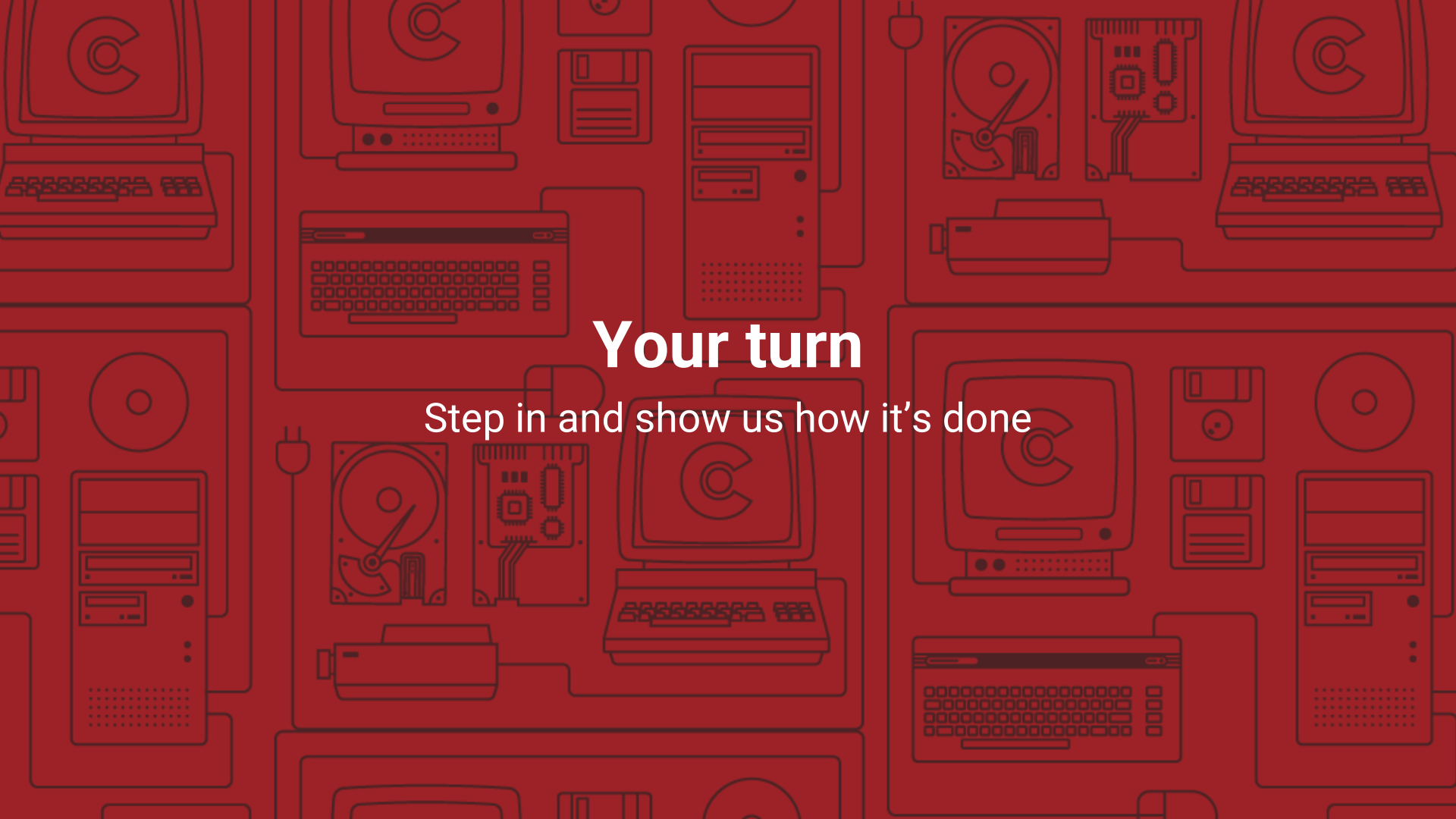
Puedes adaptar el ciclo de
decisión al dominio específico.

- Triggers configurables
- Actividades encadenadas
- Condiciones avanzadas
- Ejecución programada
- Monitoreo integrado

Un agente piensa.

Un workflow estructura cómo interactúan.





Your turn

Step in and show us how it's done